CODEVALUE
School Of Technology

Lab 2.1 - Risk map from the outside in (Testers)

Goal

Find where this old code base is most likely to hurt a patient or a release, back every risk with evidence, and turn the top risks into exploratory test charters. Then run one charter.

You're done when

- ☐ `docs/notes/risk-map.md` ranks your top 8 risks. Each one has a code location, a test status and a way to reproduce it
- ☐ You checked the graph's "most connected" classes against the real tests
- ☐ You wrote 3 exploratory charters
- ☐ You ran one charter for 10 minutes and wrote down at least one finding

Option A - Guided

0. Get the code graph (2 min). Pick one:

```
pip install graphifyy
graphify update .
```

No Python, or not allowed to install? Copy the pre-built graph instead:

```
git checkout lab/2.1-graph -- graphify-out
git reset -q graphify-out
```

Add the line `graphify-out/` to `.git/info/exclude` so you don't commit it. Open `graphify-out/graph.html` in a browser and look around for 2 minutes.

1. Hot spots vs. tests (10 min). Ask mode:

```
Read graphify-out/GRAPH_REPORT.md. Take the 10 most connected classes ("god nodes").
For each one, find the tests that exercise it in src/OrganLens.Api.Tests,
src/OrganLens.Legacy.Core.Tests and tests/e2e. Cite test names.
Write NONE when there is no test. Mark tests that are skipped or ignored.
Answer as a table: class | tests | status.
```

Check 2 rows yourself: open the test files and confirm.

2. Where old and new disagree (8 min). Ask mode:

```
The legacy desktop app and the web API both analyse studies.
Find cases where they give a different result or behaviour for the same input.
For each: the input (study id or request), what legacy does, what web does, file + method for
both.
Do not guess numbers. Show the formula you used.
```

Recalculate at least one number yourself. SYN-0002 is 110 × 50 × 45 mm.

3. Rank the risks (10 min). Agent mode:

```
Create docs/notes/risk-map.md with a table:
Rank | Area | What can go wrong | Evidence (file + method) | Tested? (test name or NONE) | Who
gets hurt | How to reproduce
Use what we found in this chat. Rank by impact on a patient report first, then by likelihood.
Maximum 8 rows. Every row needs a reproduction step a tester can follow.
```

Read every row. Delete any row whose reproduction step you could not follow.

4. Charters (5 min). Ask mode:

```
Turn the top 3 risks in docs/notes/risk-map.md into exploratory test charters.
Format: "Explore <area> with <resources> to discover <information>".
Add for each: 3 test ideas and the data to use (synthetic SYN-xxxx studies only).
```

Rewrite each charter in your own words, short. Add them to the bottom of `risk-map.md`.

5. Run one charter (10 min). Start the app:

```
dotnet run --project src/OrganLens.Api
```

Open `http://localhost:5080`, or use `src/OrganLens.Api/OrganLens.Api.http`. Run one charter for 10 minutes, without AI. Write notes: what you tried, what you saw, what surprised you.

> Visual Studio: open `OrganLens.Api.http` and use **Send Request** above each request.

Option B - Challenge

Same results, your own prompts. Rules:

- The graph is your starting point. You may not open a source file until you've read `GRAPH_REPORT.md`.
- At least one risk must come from the legacy desktop app, one from the web API, and one from the gap between them.
- At least one risk must be a security or privacy risk, with the exact line.
- For each risk, say whether a unit, integration, e2e or manual test is the cheapest way to cover it, and why.

Stop and check

- Which god node has no tests at all? Is that a problem or not?
- Which of your risks did the graph point you to, and which did it not see at all?
- If you had one day of testing before a release, which charter would you run first?